AGENTIC AI SYSTEMS · LECTURE 3

The Evolution of AI Benchmarking

From static tasks to interactive agents

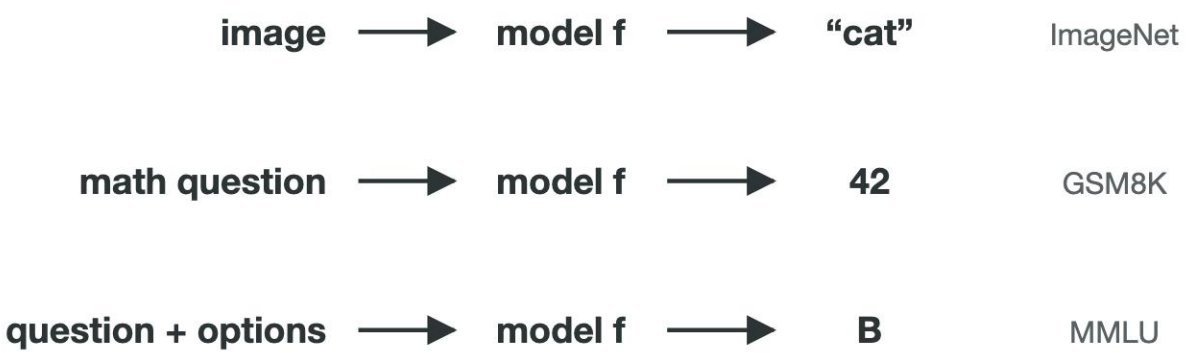
Yonathan Efroni
Tel Aviv University

PART I

The evolution of AI benchmarking

Traditional benchmarks: predict the correct answer

A fixed test set $\mathcal{D}_{\text{test}} = \{(x_i, y_i)\}_{i=1}^{n}$; score predictions $\hat{y}_i = f(x_i)$ against references.



$$\widehat{\text{Acc}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{\hat{y}_i = y_i\}$$

Standardized and reproducible — but they measure a correct *output*, not successful *behavior*.

From predicting answers to completing tasks

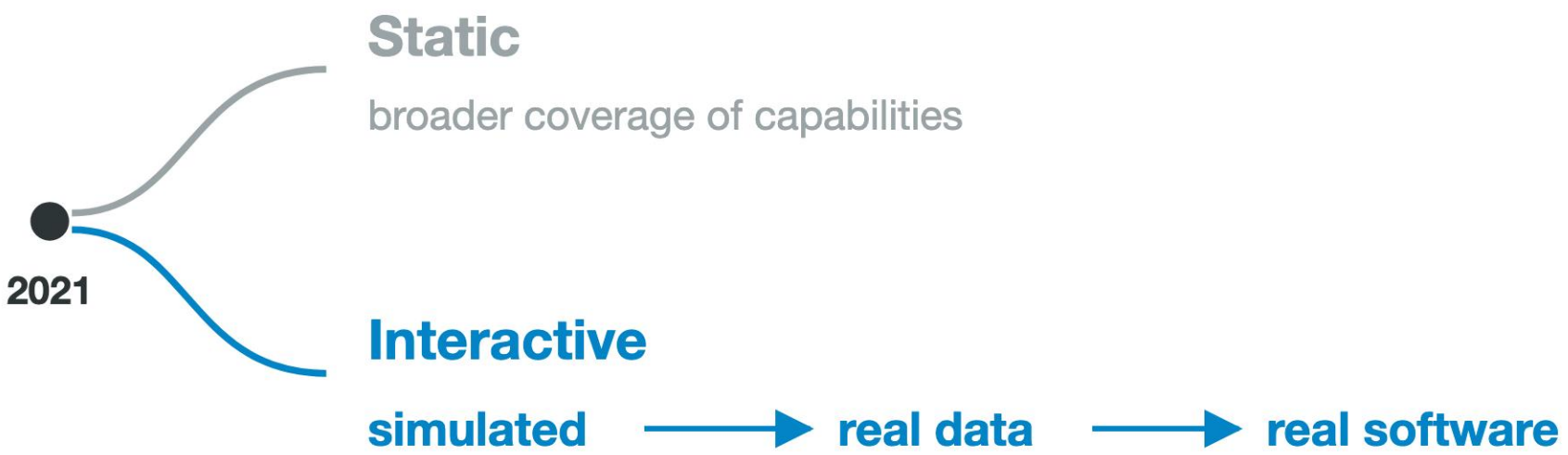
- **Static:** score one prediction per input.
- **Interactive:** score an episode of reasoning, tool use and actions.

Static	Interactive
$x \rightarrow f(x) \rightarrow \hat{y}$	$o_0 \rightarrow a_0 \rightarrow o_1 \rightarrow \cdots \rightarrow o_T$
one input, one output, one reference	a sequence of actions that changes the world

An interaction needs an environment, a protocol and a success condition — Lecture 2's design choices.

Two directions of expansion

Our focus today: interactive benchmarks, which move from simulation toward real software.



Modern AI benchmarks, 2021 and beyond

	2021	2022	2023	2024	2025	2026	2027+
Static capability	MMLU GSM8K	BIG-bench HELM					
Interactive software agents		WebShop	AgentBench GAIA **SWE-bench** **WebArena**	OSWorld τ-bench AppWorld MLE-bench	Terminal-Bench RE-Bench PaperBench BrowseComp	Agents' Last Exam SciAgentArena EdgeBench	?
Robotics / embodied	ALFWorld		**LIBERO**				

Diverse problems, simplified environments? | Realistic tasks, controlled environments? | Complex tool use, reliability? | Autonomous, long-horizon, economically meaningful work?

How do we analyze a modern benchmark?

We use the design principles of Lecture 2 as a lens.

1	**Task data**	Where do the tasks come from?
2	**Environment**	What does the agent interact with?
3	**Verification**	How is success decided?
4	**Protocol & metrics**	Which agentic system is evaluated? What are the metrics?
5	**Maintenance**	Are the tasks diverse? Do they cover what matters?

Case studies

- **SWE-bench** — software engineering
- **WebArena** — web interaction
- **LIBERO** — robot learning

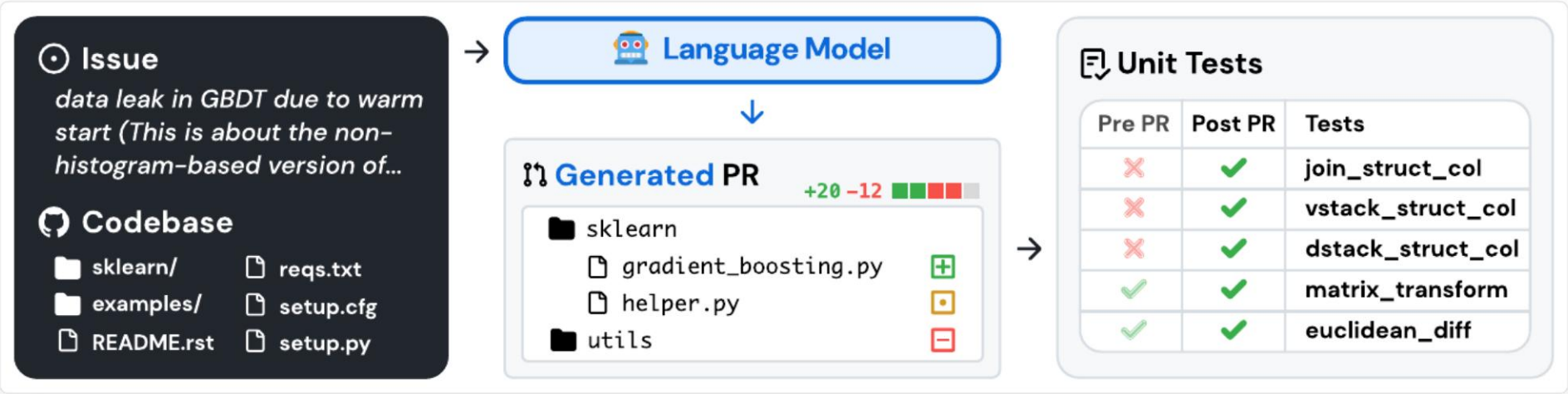
CASE STUDY 1

SWE-bench: real-world software engineering

What does SWE-bench evaluate?

Idea: turn real GitHub issues and their tests into automatically checkable tasks.

Input: repository at a fixed revision + issue text. **Output:** a patch. Tests stay hidden.



Jimenez et al., *SWE-bench* (ICLR 2024), Fig. 1 (CC BY 4.0)

Typical task: a 140-word issue in a codebase of ~1,900 files and 400K lines (medians).

Task construction: from GitHub history to tasks

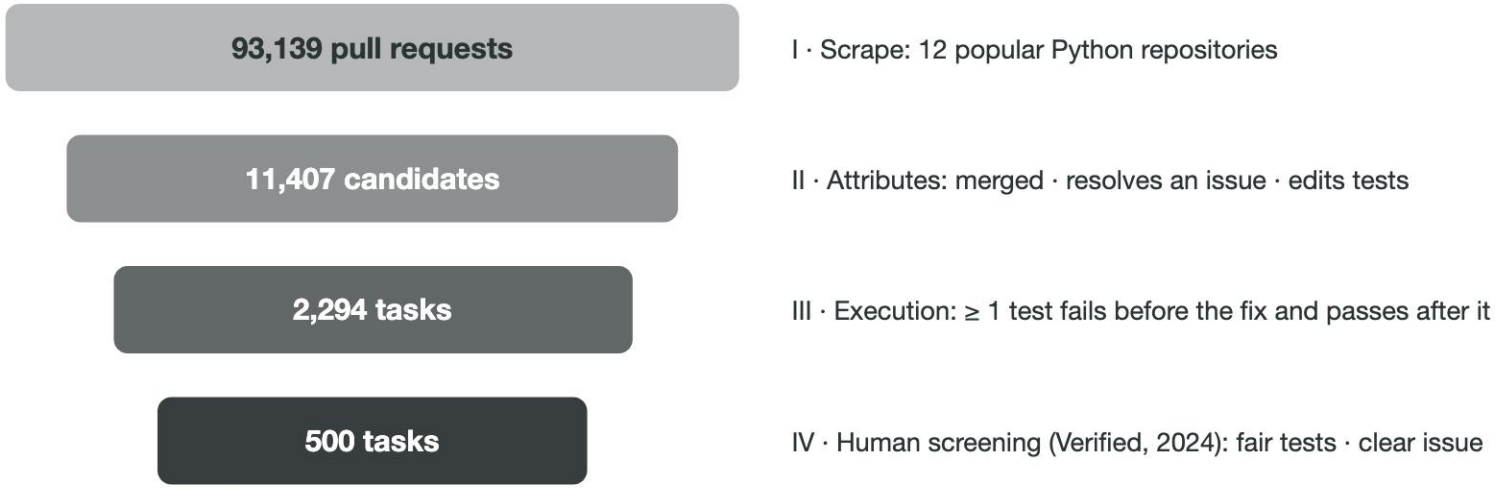
Each task is rebuilt from a merged pull request that fixed a real issue.



- **Task input:** the repository before the fix + the issue text.
- **Grader:** the tests the developers added in the same pull request.

Filtering: from pull requests to tasks

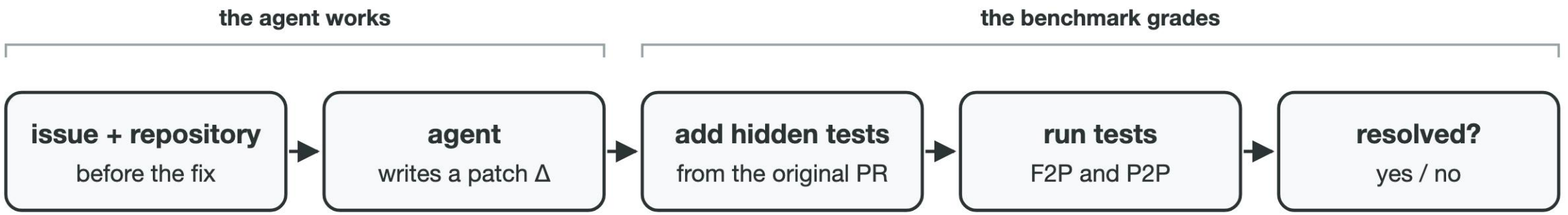
Keep only pull requests that become reproducible, checkable tasks.



What stages I–III miss: whether the tests are *fair* — tests written for the developers' fix can reject a different, correct fix. This is why stage IV was added.

How an agent's solution is graded

The grader is the original pull request's own test patch, applied after the agent's patch.



$$\text{resolved} \iff \text{all F2P tests pass} \land \text{all P2P tests pass}$$

- **Fail-to-pass (F2P):** tests for the issue — fail on the buggy code, pass on the fixed code.
- **Pass-to-pass (P2P):** existing tests — pass before and must still pass, so nothing else broke.

Agentic system, metrics and coverage

SWE-bench fixes the tasks and the metric; it leaves the agentic system open.

4 · System and metrics

- **System:** any model + scaffold + tools
- **Metric:** % of issues resolved — all F2P and P2P tests pass

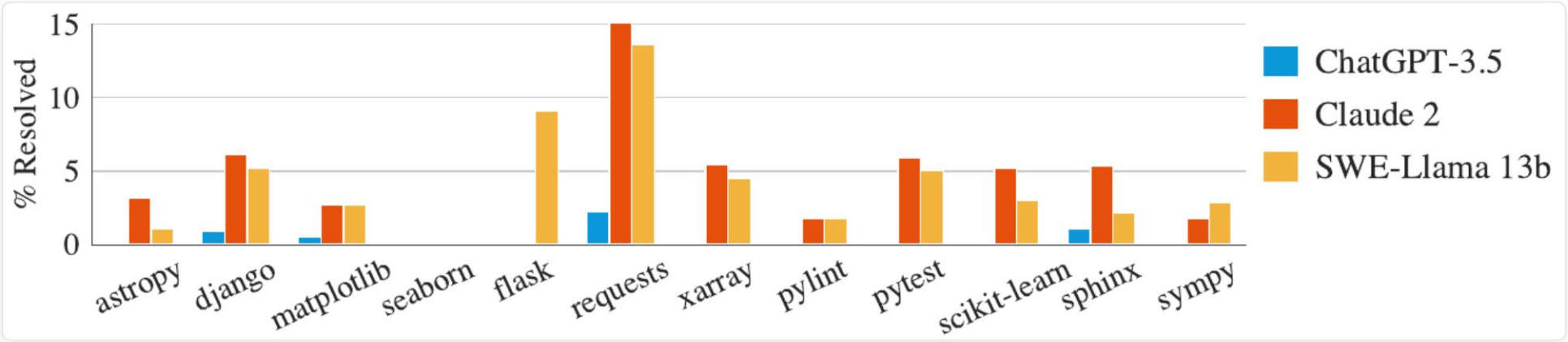
5 · Diversity and coverage

- Python only, 12 popular repositories
- django alone: 850 of 2,294 tasks
- Coverage is narrow: few repositories, one language

Same model, different scaffolds: GPT-4 on SWE-bench Lite ranges from 2.7% to 28.3%.

SWE-bench: summary

The authors approximate "the intended fix" with a measurable quantity: passing the developers' tests.



SWE-bench, Fig. 4 — % resolved per repository (oracle retrieval)

If an agent passes all tests, has it correctly resolved the issue?

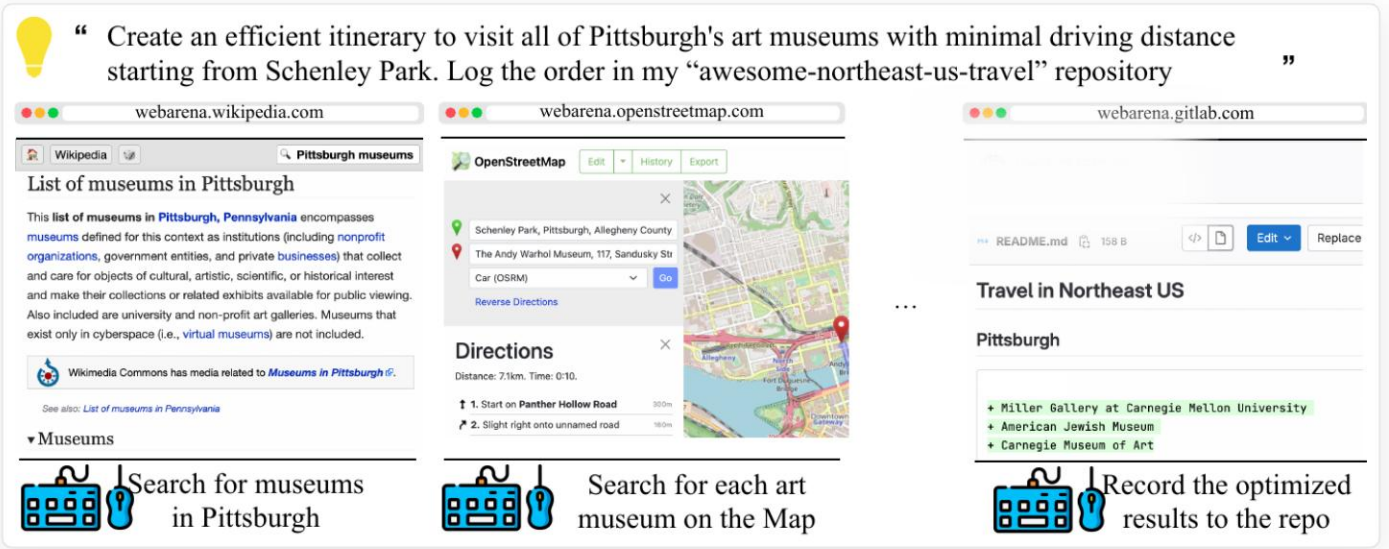
CASE STUDY 2

WebArena: interactive web agents

What does WebArena evaluate?

Idea: self-host real web apps so agents can be tested realistically and reproducibly.

Input: a goal in natural language. **Output:** browser actions — the result is often a change to the site, not a sentence.



Zhou et al., *WebArena* (ICLR 2024), Fig. 2

Task construction: templates written by the authors

Authors write task templates and fill them with different values.

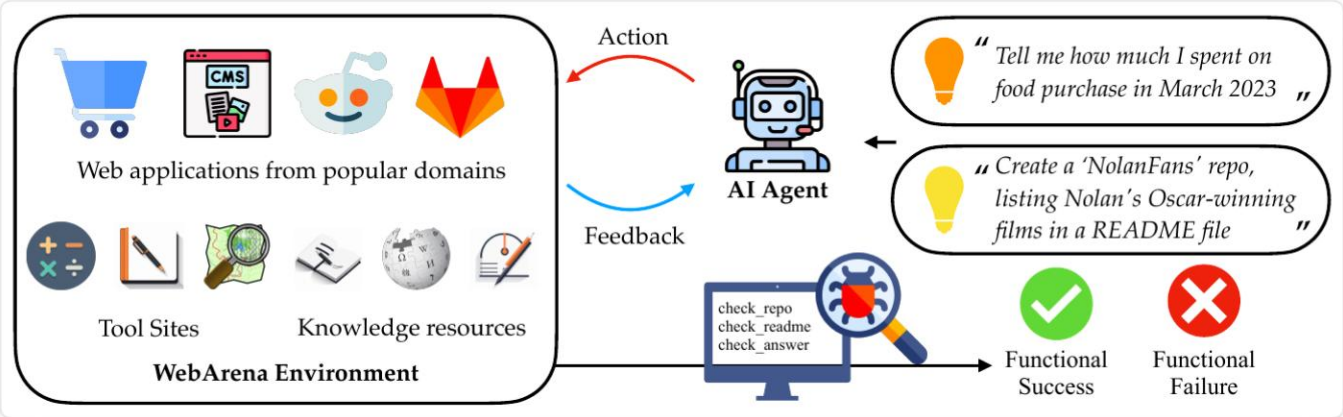
- **241 templates → 812 tasks**
- 3 categories: information seeking, site navigation, content and configuration
- Some tasks cannot be done on the sites — the agent must recognize this and answer "N/A" instead of guessing

Category	Example
Information Seeking	When was the last time I bought shampoo
	Compare walking and driving time from AMC Waterfront to Randyland
Site Navigation	Checkout merge requests assigned to me
	Show me the ergonomic chair with the best rating
Content & Config	Post to ask "whether I need a car in NYC"
	Delete the reviews from the scammer Yoke

WebArena, Fig. 5

The environment: self-hosted websites

An **isolated sandbox**, not the live web: real open-source web apps in Docker, reset before every run.



WebArena, Fig. 1

Why not live websites? Content changes, logins and CAPTCHAs — and no way to reset or inspect the state. Even the map and Wikipedia are offline copies.

How an agent's solution is graded

Check whether the goal was reached — not whether a fixed path was followed.

	Task given to the agent	Agent's result	How it is checked
Question	"Tell me the name of the customer who has the most cancellations"	an answer, e.g. "Samantha Jones"	answer equals the reference "Samantha Jones"
Change to the site	"Post to ask whether I need a car in NYC"	a new post on the forum	read the latest post: is it in the NYC forum, and does it mention "a car in NYC"?

Examples from WebArena, Table 1

Agentic system and metrics

The agent sees the page, takes one browser action at a time, and stops when done.

$$\tau = (o_0, a_0, o_1, a_1, \ldots, o_T)$$

- **Observation:** screenshot, HTML or accessibility tree
- **Actions:** click, type, scroll, open tab, go to URL, stop
- **System:** a prompted LLM agent, at most 30 steps
- **Metric:** % of tasks completed

Diversity and coverage

Realistic workflows, but on a small, fixed slice of the web.

- 4 websites; 812 tasks from only 241 templates
- Not covered: the open web — logins, pop-ups, pages that change
- Coverage is narrow: few sites, tasks that share templates

WebArena: summary

A sandboxed web environment built for evaluation; "completing a task" is approximated by a check on the site's final state.

CoT	UA Hint	Model	SR	SR_{AC}	SR_{UA}
✓	✓	TEXT-BISON-001	5.05	4.00	27.78
✗	✓	GPT-3.5	6.41	4.90	38.89
✓	✓	GPT-3.5	8.75	6.44	58.33
✓	✓	GPT-4	11.70	8.63	**77.78**
✗	✗	GPT-3.5	5.10	4.90	8.33
✓	✗	GPT-3.5	6.16	6.06	8.33
✓	✗	GPT-4	**14.41**	**13.02**	44.44
-	✓	Human	78.24	77.30	100.00

WebArena, Table 2 — % of tasks completed; best GPT-4 14.4%, humans 78.2%

If an agent completes 90% of WebArena tasks,
can it complete 90% of arbitrary web tasks?

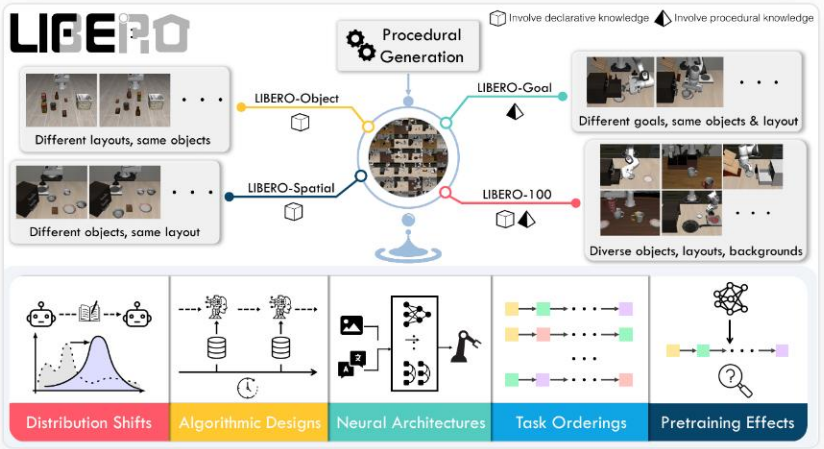
CASE STUDY 3

LIBERO: robotics and knowledge transfer

What does LIBERO evaluate?

Idea: in simulation, vary one kind of knowledge at a time to measure transfer in lifelong robot learning.

Input: a language instruction + camera images. **Output:** robot arm actions, task after task.



Liu et al., *LIBERO* (NeurIPS 2023), Fig. 1 (CC BY 4.0)

*Kinds of knowledge: **where** things are · **what** they are · **what to do** with them.*

The setup: continual learning

Tasks $\mathcal{T}_1, \ldots, \mathcal{T}_K$ arrive in sequence; task $\mathcal{T}_\tau$ comes with demonstrations D_τ (50 per task).

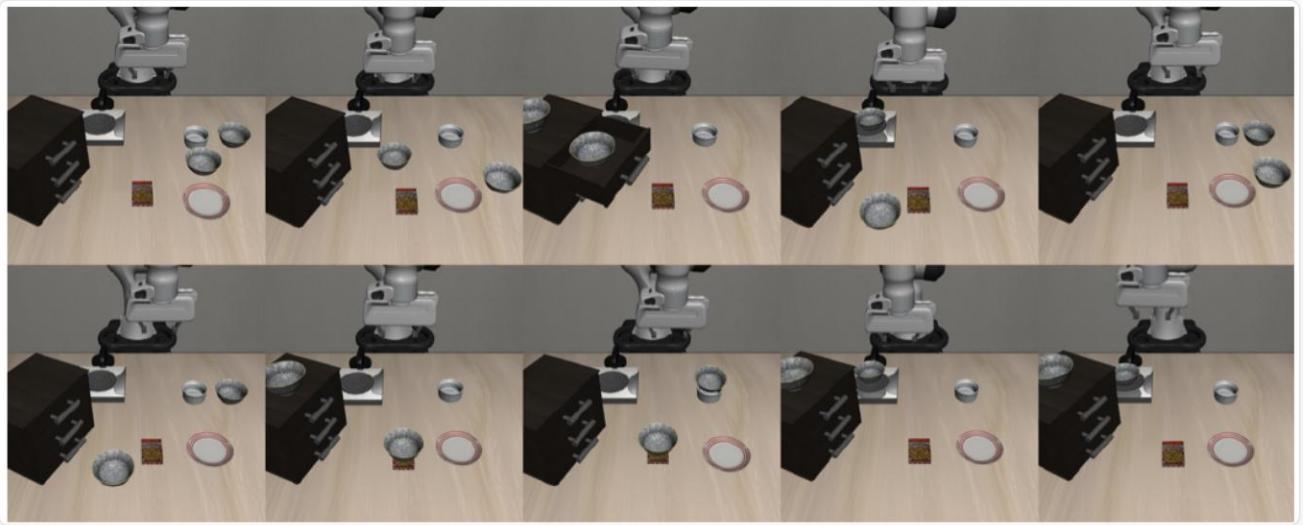
At each step $\tau = 1, \ldots, K$:

- **Learn:** $\pi_\tau = \mathcal{A}(\pi_{\tau-1},\, D_\tau)$
- **Evaluate** on all tasks up to and including τ: $c_{\tau,k} = \text{success of } \pi_\tau \text{ on } \mathcal{T}_k,\ k \le \tau$

$\mathcal{A}$ sees only D_τ — plus whatever it chooses to store from earlier tasks.

Task construction through controlled variation

Example, LIBERO-Spatial: the same objects in every task — only where they are placed changes.

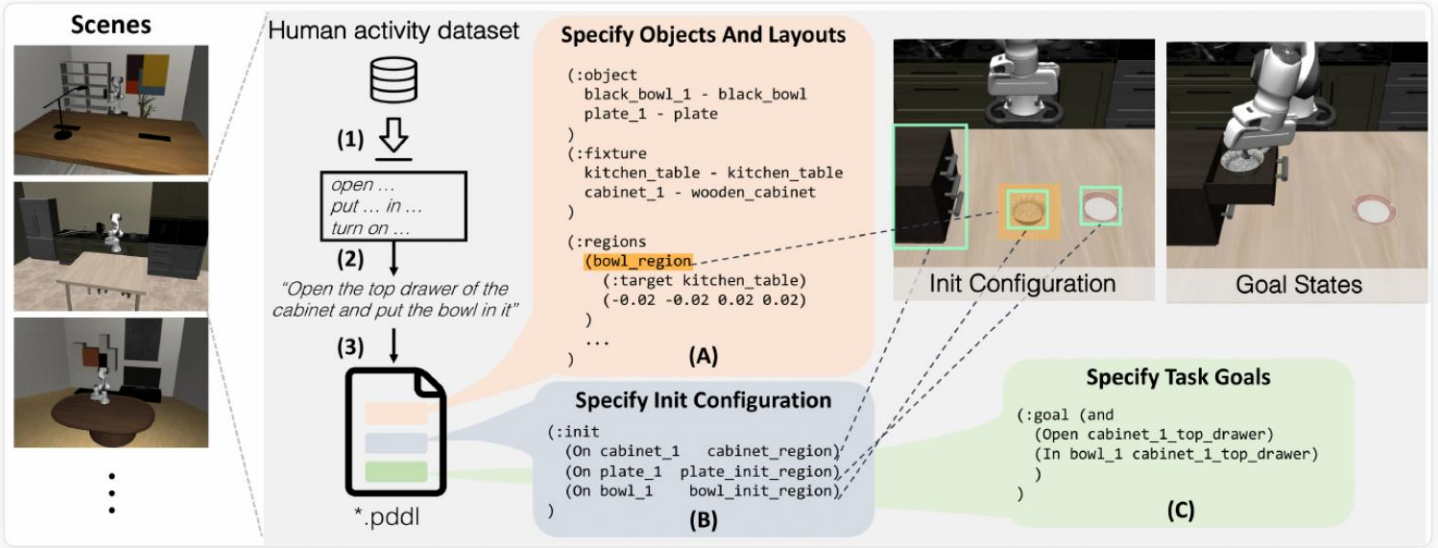


LIBERO-Spatial: ten tasks, ten layouts (paper Fig. 8)

Other suites vary the objects or the goal; LIBERO-100 mixes all factors.

The environment: a robot simulator

A **physics simulator** built for this benchmark: one robot arm, tabletop scenes, reset to a sampled start state.



LIBERO, Fig. 2 — scene, objects and start state specified for the simulator

Policies learn from 50 human demonstrations per task.

How an agent's solution is graded

An episode succeeds once all goal conditions hold in the simulator.

- **Goal:** logical conditions on objects, e.g. On(bowl, plate) and Open(drawer)
- **Check:** the simulator evaluates them at every step; up to 600 steps
- **Repeat:** 20 rollouts per task; success rate = fraction that succeed

Key metrics: learning and forgetting

Both are read off the matrix $c_{\tau,k}$ — success on task k after learning task τ.

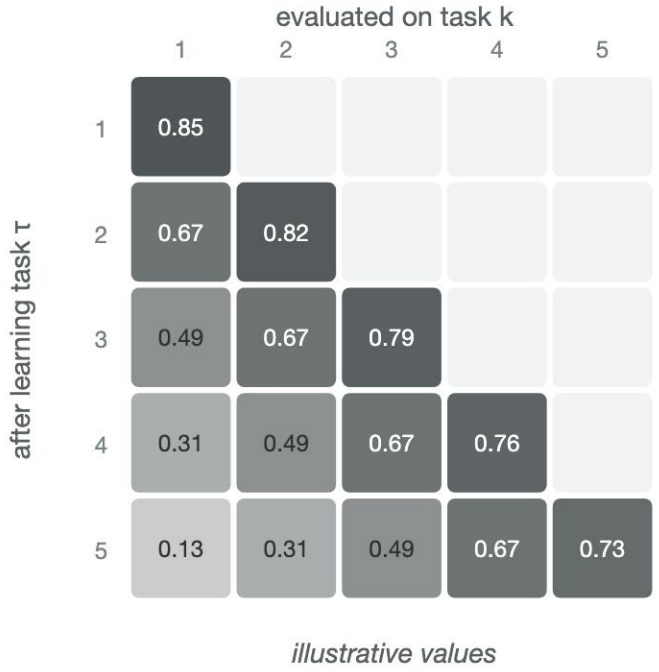
Learning (FWT, simplified): success on each new task when it is learned — the diagonal.

$$\mathrm{FWT} = \frac{1}{K}\sum_{k=1}^{K} c_{k,k}$$

Forgetting (NBT): how much success on a task drops after learning later tasks — below the diagonal.

$$\mathrm{NBT} = \frac{1}{K}\sum_{k=1}^{K} \frac{1}{K-k}\sum_{\tau>k}\left(c_{k,k} - c_{\tau,k}\right)$$

Higher FWT is better; lower NBT is better.



Diversity and coverage

Controlled variation of tabletop manipulation, in simulation.

- 130 tasks: spatial, object and goal variation, plus long tasks
- Not covered: real robots, real physics, tasks beyond the tabletop
- Coverage is narrow: one simulated arm, one kind of scene

LIBERO: summary

The authors approximate "lifelong learning" with measurable quantities: success on new and earlier tasks after each stage.

Lifelong Algo.	FWT(↑)	NBT(↓)	AUC(↑)	FWT(↑)	NBT(↓)	AUC(↑)
	LIBERO-LONG			LIBERO-SPATIAL		
SEQL	**0.54** ± 0.01	0.63 ± 0.01	0.15 ± 0.00	**0.72** ± 0.01	0.81 ± 0.01	0.20 ± 0.01
ER	0.48 ± 0.02	0.32 ± 0.04	**0.32** ± 0.01	0.65 ± 0.03	0.27 ± 0.03	0.56 ± 0.01
EWC	0.13 ± 0.02	0.22 ± 0.03	0.02 ± 0.00	0.23 ± 0.01	0.33 ± 0.01	0.06 ± 0.01
PACKNET	0.22 ± 0.01	**0.08** ± 0.01	0.25 ± 0.00	0.55 ± 0.01	**0.07** ± 0.02	**0.63** ± 0.00
MTL			0.48 ± 0.01			0.83 ± 0.00

LIBERO, Table 2 (two of four suites; AUC combines FWT and NBT) — SeqL learns fastest but forgets most

Two policies have the same average success rate. Are they equally good lifelong learners?

PART V

Conclusion

What makes a benchmark meaningful?

A score is evidence only within its tasks, environment, verifier and protocol.

$$\text{high benchmark score} \not\Rightarrow \text{real-world competence}$$

What should we evaluate in next-generation agents that today's benchmarks miss?